

1주

인공지능과 프로그래밍



1주

학습 목표

- **AI시대의 프로그래밍 능력**
- **소프트웨어와 컴퓨터 프로그래밍**
- **인공지능과 프로그래밍**

AI시대의 프로그래밍 능력



- AI시대 프로그래밍 능력이 왜 필요한가?



<https://www.youtube.com/watch?v=sf52u9ZvKzk&t=8s>



"AI 도입으로 코드 생성 비용이 급격히 낮아지면서 (...) 리뷰와 검증 단계가 가장 중요한 병목 구간으로 부상했다."

2월 들어 클로드의 새로운 기능과 AI 에이전트들이 스스로 코드를 고치는 모습을 보면서, 주변에서 "이제 소프트웨어의 시대가 끝나는 거 아니냐"는 무거운 이야기가 들려왔어. 덩달아 뭔가 무거운 긴장감이 느껴지는 요즘이야. 하지만 너무 걱정만 하며 웅크리고 있을 필요는 없지. 위기가 곧 기회라는 말은 이럴 때 쓰는 말일 테니, 오히려 숨겨진 기회를 찾고 '진짜 실력'이 뭔지 보여줄 수 있는 찬스의 시간이 될 수 있겠다는 생각을 해봤어. 그럼 그 기회란 것 어디서 찾아야 할까?

AI 시대에 IT 업계에서 가장 핫한 키워드는 '바이브 코딩'이라고 생각해. 예전처럼 한 땀 한 땀 로직을 짜는 게 아니라, 자연어로 "이런 느낌의 기능을 만들어줘"라고 말하면 AI가 짤하고 코드를 만들어주는 방식 말이야. (전)테슬라의 안드레이 카르파티도 프로그래밍 시간의 80%를 영어로 소통한다고 하니, 세상이 참 빠르게 변하고 있단 말이지.

하지만 여기에는 반전이 숨어 있어. AI가 코드를 1초 만에 뱉어내다 보니, 검토해야 할 코드가 산더미처럼 쌓여버리는 거야. 코드 리뷰에 드는 시간이 무려 90%나 늘어났다는데, AI가 짤 똥(?)을 치우느라 정작 기운을 다 빼고 있는 셈이지. 특히 무서운 건 '이해 부채(Comprehension Debt)'라는 녀석이야. AI가 준 코드가 당장 돌아간다고 해서 '에라 모르겠다'하고 넘어가면, 나중에 그 코드가 왜 돌아가는지 아무도 모르는 상태가 되거든. 이건 냅이 마음을 다 안다고 착각하다가 갑자기 냅펀치를 맞는 것(?)만큼이나 위험한 일이야 (!!!). 시스템이 거대한 '블랙박스'가 되어가고 있다는 소리니까.

결국 지금 같은 환경에서 살아남는 전략은 코드를 많이 만드는 게 아니야. '어떤 코드가 좋은지 알아보는 안목'과 'AI가 만든 결과물을 철저하게 검증하는 능력'이 핵심이 아닐까. 기술이 우리를 대신하는 게 아니라, 우리가 AI라는 강력한 엔진을 조종하는 아키텍트가 되어야 한다는 뜻이기도 해.

AI 시대가 왔다고 SW 종말론 같은 소릴 들을 게 아니라, 오히려 '실제 역량'이 더 귀해지는 시대가 온 거라고 생각해 보면 어떨까. AI가 쏟아내는 정보의 파도 속에서 어떻게 중심을 잡고 '이해 부채'를 관리해야 할지 기존과는 조금은 다른 관점에서 접근해보기도 하고, 고민의 방향을 바꿔보는 것도 좋을 것 같아.

■ AI시대의 프로그래밍 능력



- AI를 **활용**할 수 있는 능력
- AI가 생성한 **결과**를 **검증**할 수 있는 능력
- AI에게 **방향**을 제시하고, AI를 **이끌어 원하는 결과**를 **도출**할 수 있는 능력



AI 시대의 지식의 관점

공학/인문학/의학/법학 등 전분야

어떻게 지시를 내릴까?



어떻게 활용할까?

지식습득/ 학습 / 분석/ 의사결정 . etc



ChatGPT

파이썬으로 주사위게임 코드 생성해줘



물론이죠! 간단한 주사위 게임을 만들어 보겠습니다. 이 게임에서는 두 명의 플레이어가 주사위를 굴려서 높은 숫자를 낸 플레이어가 이기는 구조입니다. 아래는 Python으로 구현한 주사위 게임 코드입니다:

```
def roll_dice():  
    return random.randint(1, 6)  
  
def play_game():  
    print("주사위 게임에 오신 것을 환영합니다!")  
  
    input("플레이어 1, 주사위를 굴리려면 Enter를 누르세요...")  
    player1_roll = roll_dice()  
    print(f"플레이어 1이 굴린 주사위: {player1_roll}")  
  
    input("플레이어 2, 주사위를 굴리려면 Enter를 누르세요...")  
    player2_roll = roll_dice()  
    print(f"플레이어 2가 굴린 주사위: {player2_roll}")  
  
    if player1_roll > player2_roll:  
        print("플레이어 1이 이겼습니다!")  
    elif player1_roll < player2_roll:  
        print("플레이어 2가 이겼습니다!")  
    else:  
        print("무승부입니다!")  
  
if __name__ == "__main__":  
    play_game()
```

프롬프트를 어떻게 잘 만들어
명령을 내릴까?

이 코드를 어떻게 활용할까

소프트웨어와 프로그래밍 언어

소프트웨어의 역할

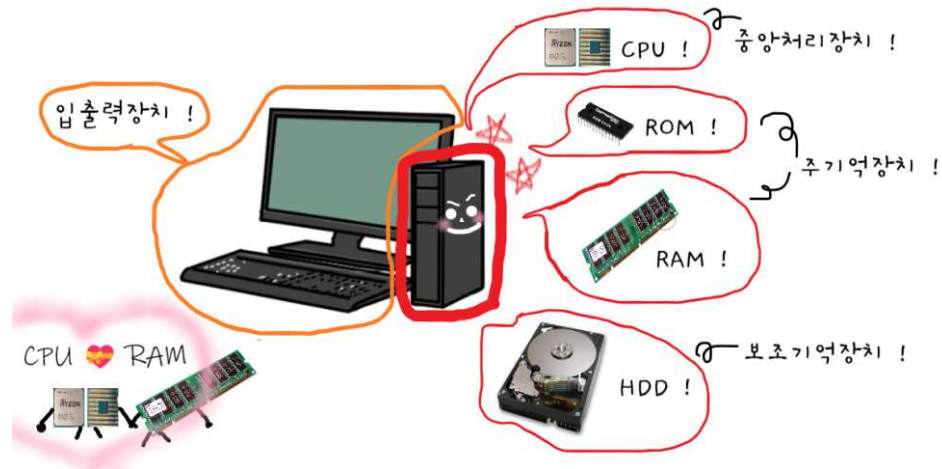


■ 컴퓨터의 구성

- 하드웨어 + 소프트웨어

■ 하드웨어

- 컴퓨터를 구성하는 기계 장치
- 중앙처리장치(CPU), 기억장치, 입력장치, 출력장치 등이 있음



■ 소프트웨어

- 하드웨어를 작동시키기 위한 **명령어의 집합**
- 보통 **프로그램**과 혼용하여 불림
- 소프트웨어가 하드웨어를 작동시키기 위해서는 **먼저 메모리에 탑재됨**
- 프로그램이 메모리에 탑재되고 나면 **실행**되는데, 이 단계에서 프로그램은 **기계어로 변환되어 각종 하드웨어를 작동시킴**

$x = 10 + 2$
 $y = x + 4$
[처리내용 예]



001001 11101 11101 1111111111111000
001000 00001 00000 0000000000001010
001000 00001 00001 0000000000000010
101011 11101 00001 0000000000000000
001000 00010 00001 0000000000000000
101011 11101 00010 0000000000000000
001001 11101 11101 0000000000001000

기계어



■ 프로그래밍 언어의 종류

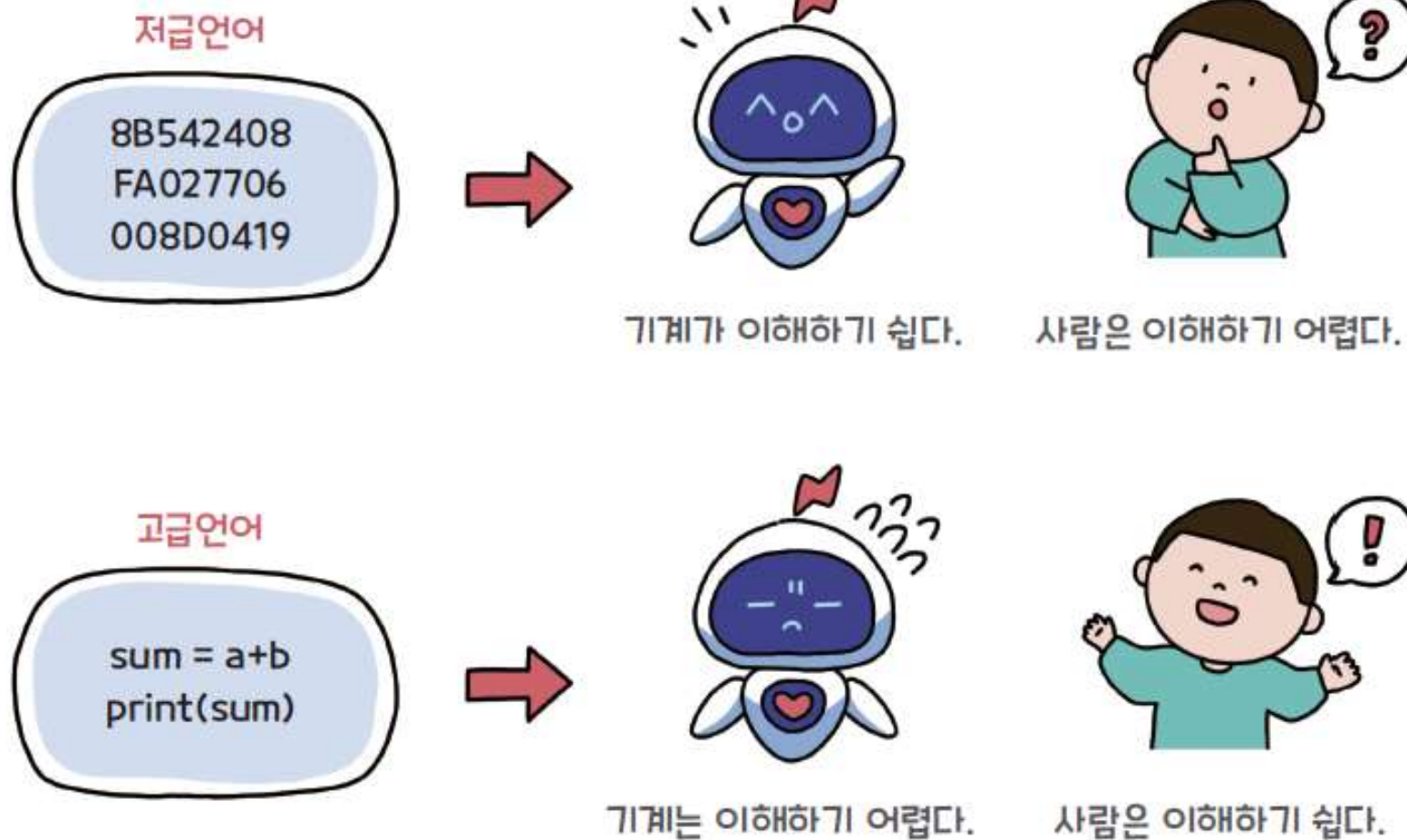


그림 1-6 저급언어와 고급언어



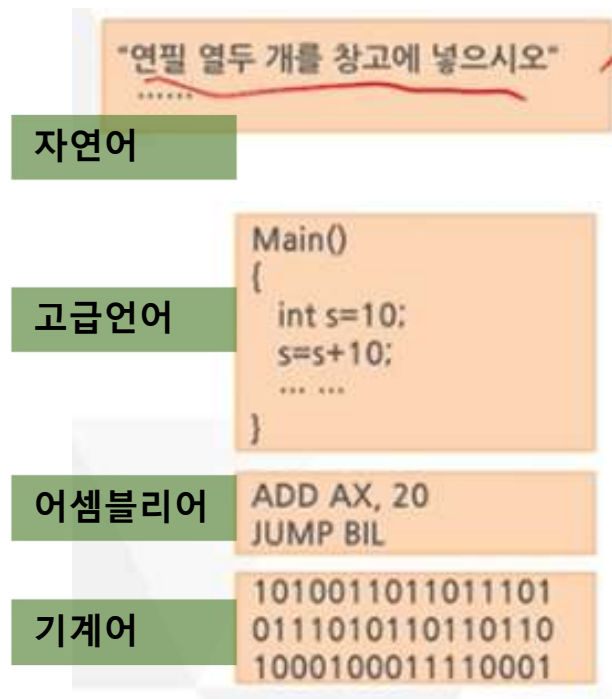
■ 프로그래밍 언어의 종류

■ 저급 언어

- 컴퓨터가 쉽게 이해할 수 있는 언어로 실행 속도가 빠르고 성능이 뛰어남
- 그러나 배우기 어렵고 유지보수가 힘들
- 기계어, 어셈블리어 등

■ 고급 언어

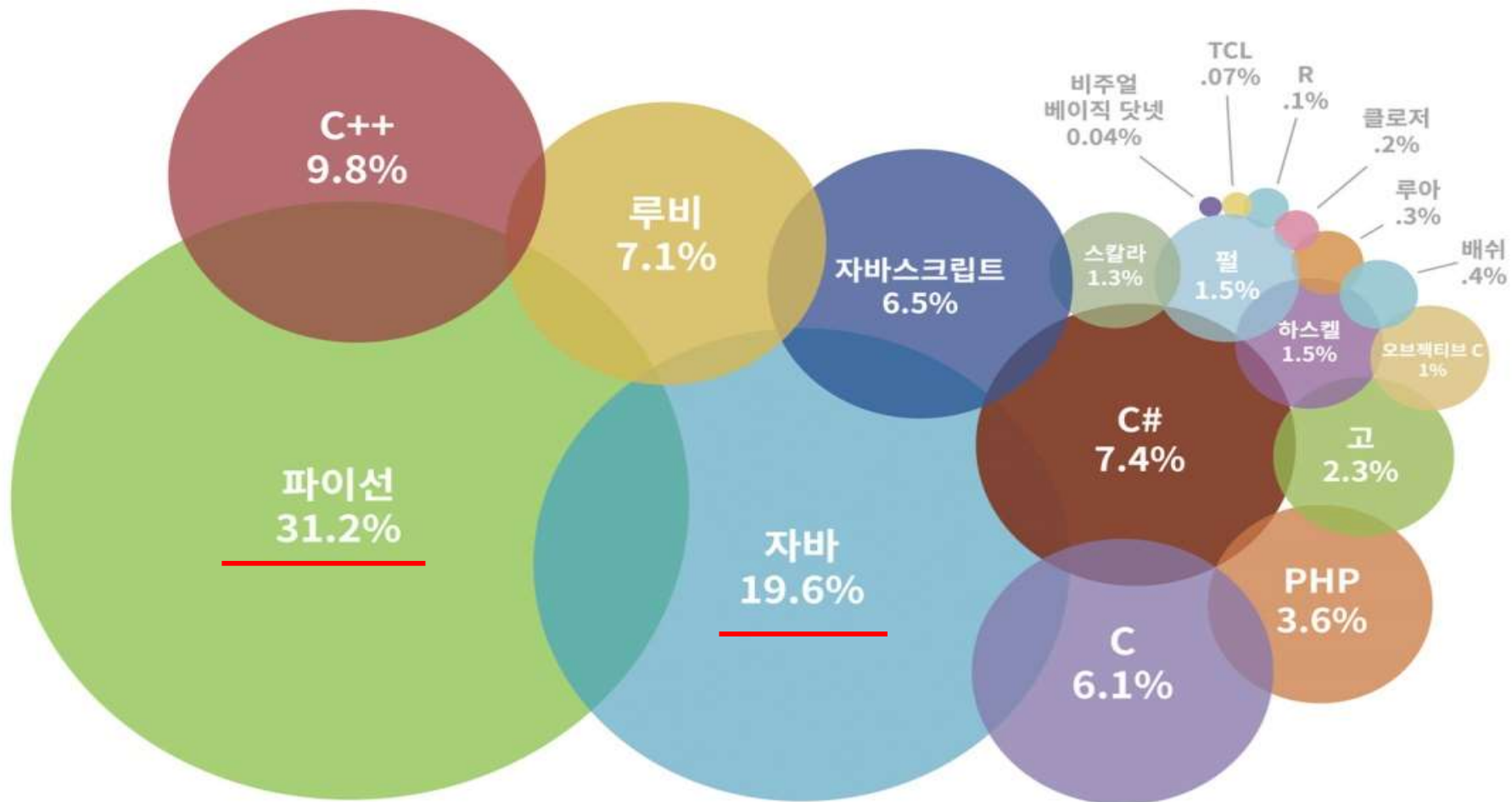
- 인간이 사용하는 언어와 유사하게 만들어진 언어
- 배우기 쉽고, 코드 판독이 쉬워 유지보수가 용이
- C, C++, C#, 자바, 자바스크립트, 파이썬 등





■ 프로그래밍 언어의 종류

가장 인기있는 프로그래밍 언어





■ 소스코드

- C, 자바, 파이썬 같은 프로그래밍 언어로 작성한 프로그램 코드
- 사람이 판독할 수 있는 **고급언어**로 작성되어 **텍스트 형태의 파일**로 저장됨
- 하나의 소프트웨어는 **하나 또는 그 이상의 소스코드 파일**로 구성됨

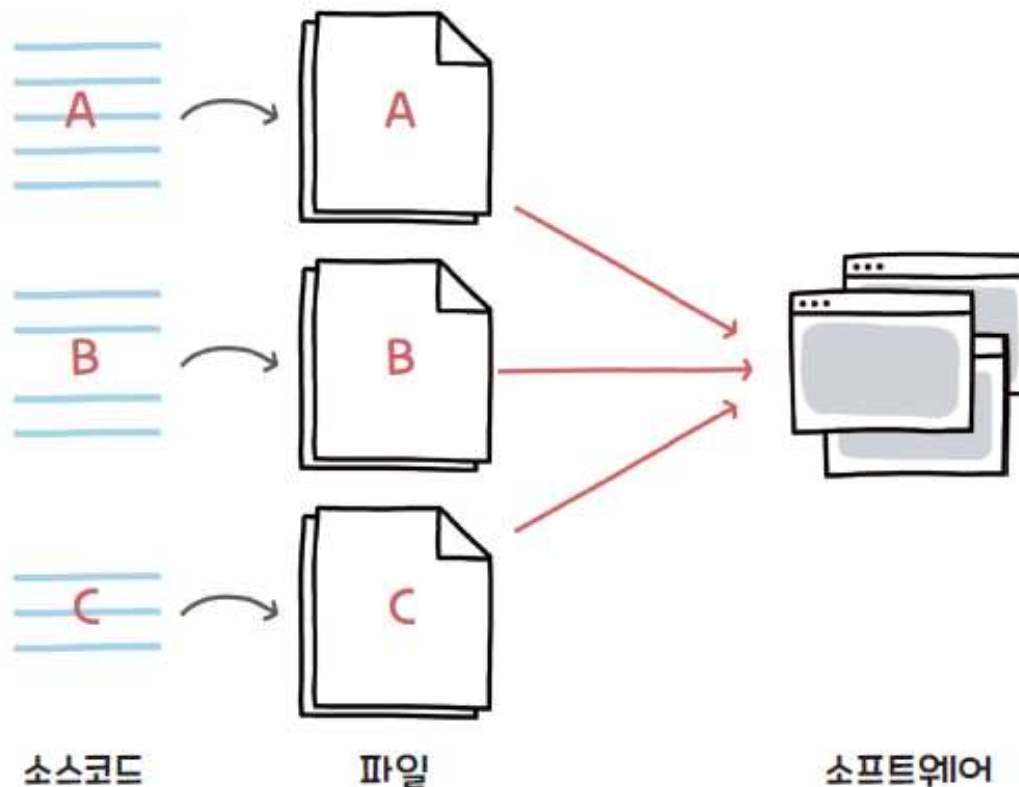


그림 1-7 여러 소스코드 파일로 구성된 소프트웨어

```
1 package com.week7;
2
3 import java.util.Scanner;
4
5 interface Fruit{
6     String Won = "원";
7     public int getPrice();
8     public String getName();
9 }
10
11 interface ItemFruit extends Fruit{
12     public String getItems();
13 }
14
15 class Orange implements ItemFruit{
16     @Override
17     public int getPrice() {
18         return 1000;
19     }
20
21     @Override
22     public String getName() {
23         return "오렌지";
24     }
25 }
```

JAVA 코드
Test.java

```
print ( "Hello World" )
print ( "Welcome to Python" )
```

파이썬 코드
Hello.py



■ 목적코드

- 고급언어로 작성된 **소스코드**를 실행하면 코드는 이진수로 이루어진 **목적코드(object code)**로 변환됨
- 소스코드는 **컴파일러(compiler)**를 통해 목적코드로 변환



그림 1-8 소스코드가 목적코드로 변환되는 과정

↑
소스코드를 기계어코드로 바꿔주는 번역기

인공지능과 컴퓨터 프로그래밍



➤ 전통적 컴퓨터 프로그래밍과 인공지능의 전환점

- ✓ 전통적 컴퓨터 프로그래밍의 한계와 이해
 - **이진 데이터 처리** : 본래 컴퓨터는 0과 1의 이진 데이터를 주로 처리하는 시스템
 - **프로그래밍의 본질** : 전통적인 컴퓨터 프로그램은 사람이 알고리즘과 규칙을 직접 작성하며, 컴퓨터 내부에서는 모든 **데이터가 0과 1의 이진** 형태로 처리
- ✓ 인공지능(AI)의 등장과 패러다임의 변화
 - **빅데이터의 활용** : 인공지능이 등장하면서 **다양한 데이터를 이용하여 패턴을 학습하고 예측하는 기술**이 중요
 - **데이터 변환의 필요성** : 다양한 형태의 현실 세계 데이터를 **컴퓨터가 인식할 수 있는 형태로 처리하고 변환하는 작업**이 필수



➤ 인공지능 시대에 수학이 중요해진 이유

✓ 수학 처리의 핵심 대두

- **필수 수학 요소** : 데이터를 숫자로 표현하고 계산하기 위해 **집합, 벡터와 행렬과 같은 수학 개념**이 중요
- **알고리즘의 기반** : 선형대수(벡터·행렬), 미분, 확률과 통계 등의 수학을 기반으로 설계

✓ 수학 연구와 프로그램 활용의 역할 분담

- **이론 연구 분야** : 수학적 이론 자체를 깊이 연구하여 새로운 알고리즘을 만들어내는 전문 학문
- **실무적 활용분야** : 파이썬 등의 컴퓨터 프로그램과 라이브러리를 이용하면, 알고리즘의 내부 수학을 모두 이해하지 않아도 알고리즘의 활용법만 익혀 쉽게 사용



➤ 본 수업의 학습 목표 및 방향성

✓ 우리의 학습 관점

- **이론 중심 탈피** : 수학적 이론을 직접 증명하거나 연구하는 것이 이번 강의의 목표가 아님.
- **프로그래밍/ 라이브러리 중심 접근**: 파이썬 프로그램과 다양한 라이브러리를 활용하여, 우리가 필요한 인공지능 알고리즘을 직접 다루는 데 집중